Day 1 下午课程

Agentic AI 设计范式

从RPAR框架到工程实现
理解Agent的本质与工具演进

⏱ 120分钟

📚 3课时

🎯 RPAR框架

01

Agent本质与RPAR框架

40分钟

Agent的本质：从"动口"到"动手"

💬 传统大模型应用

❌ 只给建议，不执行

❌ 单轮对话

❌ 告诉你"怎么做"

💡 价值：信息

VS

🤖 Agent

✅ 直接执行，完成任务

✅ 多轮自主交互

✅ 直接"帮你做完"

🎯 价值：结果

核心洞察

"Agent的本质是'执行'，特别是'自主执行'——从'顾问'到'助手'"

Agentic AI 详解

与Autonomous AI的区别

Agentic AI 是一种能够自主感知环境、做出决策并执行行动的AI系统。与传统的Autonomous AI不同，Agentic AI强调"主动性"和"目标导向性"。

核心特征

- **自主性**：能够独立做出决策，无需人工干预
- **主动性**：主动探索解决问题的方法，而非被动响应
- **适应性**：根据环境变化动态调整策略
- **工具使用**：能够调用外部工具和资源完成复杂任务

与Autonomous AI的区别

维度	Autonomous AI	Agentic AI
决策方式	基于规则/预设	基于推理和学习
交互方式	被动响应	主动规划
目标达成	固定流程	动态调整

Workflow vs Agentic：从"固定"到"自主"

🚄

Workflow（高铁）

A → B → C

- ✅ 速度快，按步骤执行
- ❌ 路线固定
- ❌ 走错就停止

VS

🚗

Agentic（自驾）

起点 ⊢→ B →┐ └→ C →┘ 终点

- ✅ 可根据路况调整
- ✅ 自主规划路线
- ✅ 实时灵活应变

RPAR框架：Agent的认知架构

1

REASONING

思考

理解任务，分析现状

2

PLANNING

规划

制定执行步骤

3

ACTING

执行

调用工具执行

4

REFLECTION

反思

评估调整策略

🧠

Reasoning - 思考环节

AI理解用户意图、分析当前状况、识别约束条件和潜在风险。

- → 意图识别：理解用户的真实需求
- → 上下文理解：分析对话历史和背景信息
- → 约束分析：识别时间、资源等限制
- → 风险评估：预判可能的问题

Planning - 规划环节

基于思考结果，制定详细的执行计划，包括步骤分解和优先级排序。

- → 任务分解：将复杂任务拆分为可执行子任务
- → 优先级排序：确定执行顺序
- → 资源评估：评估所需工具和数据
- → 路径规划：选择最优执行路径

Acting - 执行环节

调用各种工具和资源，执行规划中的具体任务。

- → 工具选择：根据任务选择合适工具
- → 参数准备：准备正确的输入参数
- → 执行调用：发起工具调用
- → 结果获取：接收并解析执行结果

🔄

Reflection - 反思环节

评估执行结果，识别问题，调整策略，决定是否继续循环。

- → 结果评估：检查结果是否符合预期
- → 错误识别：发现并定位问题
- → 策略调整：修改后续执行计划
- → 知识沉淀：总结经验教训

🧠

Reasoning

3-5个思考维度：意图识别、上下文理解、约束分析...

📋

Planning

规划策略：分解、排序、资源评估

⚡

Acting

工具选择逻辑、参数准备、错误预判

🔄

Reflection

反思类型：结果评估、过程复盘、知识沉淀

三个核心突破

从黑盒到白盒

显式推理过程，可审计可追溯

从静态到动态

与环境持续交互，边做边学

从确定到适应

容错性强，可自我纠正

02

RPAR实例与Acting详解

40分钟

RPAR的裁剪：不同场景的选择

⚡ ReAct 简化版 | 📋 Plan-and-Execute 高效版 | 🎯 完整RPAR 完全体

ReAct: R-A-R 简化版

```
Reasoning → Acting → Reasoning
```

特点

✓ 简单直接，边想边做

✓ 适用于简单查询类任务

✕ 缺乏显式规划和反思

ReAct 详解

论文来源与核心思想

ReAct（Reasoning + Acting）是普林斯顿大学和Google Research在2022年提出的方法，核心思想是让AI交替进行推理和行动。

论文信息

- **论文标题**：ReAct: Synergizing Reasoning and Acting in Language Models
- **发表时间**：2022年10月
- **核心作者**：Shunyu Yao（普林斯顿），Jeffrey Zhao（Google）等
- **核心思想**：推理（Reasoning）和行动（Acting）应该相互促进

完整流程示例

ReAct循环伪代码

```
def react_loop(user_query, max_iterations=10):
    thought_history = []
    action_history = []

    for i in range(max_iterations):
        # 1. Reasoning: 基于历史和当前状态思考
        thought = llm.reason(
            query=user_query,
            history=thought_history,
            actions=action_history
        )
        thought_history.append(thought)

        # 2. Acting: 执行工具调用
        action = parse_action(thought)
        observation = execute_tool(action)
        action_history.append({"action": actio

        # 3. 检查是否完成
        if is_complete(thought, observation):
            return generate_answer(thought_his

    return "达到最大迭代次数"
```

示例：查询天气

思考→调用天气API→回答用户

Plan-and-Execute: R-P-A 高效版

```
Reasoning → Planning → Acting(List)
```

特点

✓ 先规划后批量执行

✓ 适用于步骤明确的任务

✕ 缺乏反思调整能力

示例：订机票

```
规划(查航班→选航班→预订)→批量执行
```

完整RPAR: R-P-A-R 完全体

```
Reasoning → Planning → Acting → Reflection
```

特点

✓ 完整四环节，支持试错

✓ 适用于复杂探索性任务

✕ 复杂度最高

示例：投资分析

```
多轮循环：查询→分析→反思→再查询...
```

范式选择决策树

任务复杂度?

简单明确

→ ReAct

复杂

步骤明确?

是

→ Plan-and-Execute

否

→ 完整RPAR

Acting详解：Agent的"手脚"

API调用

调用外部服务接口

查询数据库

发送通知

第三方服务

🛠️

其他工具

特殊能力调用

浏览器操作

数据库直连

命令行执行

💻

代码执行

本地代码运行

Python/JS代码

数据处理

复杂计算

📁

文件操作

读写文件系统

生成报告

保存日志

批量处理

Python API调用示例

Python

```python
import requests
from typing import Dict, Any

class PowerSystemAPI:
    """电力系统API调用工具"""

    def __init__(self, base_url: str, api_key: str):
        self.base_url = base_url
        self.headers = {
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json"
        }

    def query_investment_data(
        self,
        region: str,
        start_date: str,
        end_date: str,
        project_type: str = "全部"
    ) -> Dict[str, Any]:
        """
        查询投资项目数据

        Args:
            region: 查询区域，如'成都高新区'
            start_date: 开始日期，格式YYYY-MM-DD
            end_date: 结束日期，格式YYYY-MM-DD
            project_type: 项目类型筛选

        Returns:
            包含项目列表、投资总额等信息的字典
        """
        url = f"{self.base_url}/api/v1/investments/query
        params = {
            "region": region,
            "start": start_date,
            "end": end_date,
            "type": project_type
        }
```

```
        try:
            response = requests.get(url, headers=self.he
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e
            return {"error": str(e), "code": "API_ERROR"
```

核心洞察

"Acting让Agent从'动口'变成'动手'——没有Acting，Agent只是聊天机器人"

03

工具演进与封装

40分钟

工具演进：从API到Skills

L1	API	原始能力	各系统原始接口
L2	MCP	协议标准	统一描述和调用
L3	Tools	原子能力	具体的功能函数
L4	Code Tools	代码封装	自定义代码逻辑
L5	Skills	AI行为指导	通过Prompt定义AI的角色和工作流程

语义封装　代码封装　原子能力　标准化　原始接口

API → MCP：从"人懂"到"AI懂"

API时代

- ❌ 格式混乱，各自为政
- ❌ 文档给人看，AI读不懂
- ❌ 需要人写代码调用

❌ 难以组合和复用

人翻译 → AI

↓

MCP时代

✅ 统一标准，格式规范

✅ 自描述能力，AI能懂

✅ AI可自主选择和调用

✅ 生态互通，即插即用

AI直接理解

MCP (Model Context Protocol)

模型上下文协议详解

MCP是一种标准化协议，让AI模型能够理解工具的用途、参数和返回值。它定义了一套JSON Schema，使得工具具备"自描述"能力。

MCP的核心价值

- **第一层价值 - 描述标准化**：统一的JSON格式，让AI直接理解
- **第二层价值 - 生态标准化**：跨平台互通，一次开发多处使用

MCP字段说明

字段	类型	说明
`name`	string	工具名称，驼峰命名
`description`	string	功能描述，让AI理解用途
`parameters`	object	输入参数定义（JSON Schema）
`returns`	object	返回结果定义
`errors`	object	错误码定义

MCP描述示例 - 查询投资项目数据

JSON

💡 **MCP教学要点：** 这个JSON描述让AI"看懂"工具的能力，就像给工具配了一张"身份证+使用说明书"。

```
{
  "name": "query_investment_data",
  "description": "【这是最关键的部分！AI靠这段描述判断什么时候用
  "version": "1.0.0",
  "parameters": {
    "type": "object",
    "properties": {
      "_comment": "【输入参数定义】告诉AI：调用时需要什么数据",
      "region": {
        "type": "string",
        "description": "查询区域，如'成都高新区'、'天府新区'等
        "examples": ["成都高新区", "锦江区", "全市"]
      },
      "start_date": {
        "type": "string",
        "format": "date",
        "description": "查询开始日期，格式YYYY-MM-DD",
        "examples": ["2024-01-01", "2024-03-01"]
      },
      "end_date": {
        "type": "string",
        "format": "date",
        "description": "查询结束日期，格式YYYY-MM-DD",
        "examples": ["2024-12-31", "2024-06-30"]
      },
      "project_type": {
        "type": "string",
        "enum": ["全部", "配网改造", "变电站建设", "智能电网"
        "description": "项目类型筛选",
        "default": "全部"
      }
    },
    "required": ["region", "start_date", "end_date"],
    "additionalProperties": false
  },
  "returns": {
    "_comment": "【返回值定义】告诉AI：调用后会得到什么数据",
    "type": "object",
    "properties": {
```

```
        "projects": {
          "type": "array",
          "description": "项目列表",
          "items": {
            "type": "object",
            "properties": {
              "project_id": {
                "type": "string",
                "description": "项目唯一标识"
              },
              "project_name": {
                "type": "string",
                "description": "项目名称"
              },
              "investment_amount": {
                "type": "number",
                "description": "投资金额(万元)"
              },
              "progress": {
                "type": "number",
                "description": "项目进度(0-100)"
              }
            }
          }
        },
        "total_count": {
          "type": "integer",
          "description": "项目总数"
        },
        "total_investment": {
          "type": "number",
          "description": "投资总额(万元)"
        }
      }
    }
  }
```

MCP描述示例 - 发送通知

JSON

💡 **对比教学：** 这个工具展示MCP如何描述"操作类"能力（不是查询，而是执行动作）。

```
{
  "name": "send_notification",
  "description": "【操作类工具】向指定用户或用户组发送系统通知。
  "version": "1.2.0",
  "parameters": {
    "type": "object",
    "properties": {
      "channel": {
        "type": "string",
        "enum": ["dingtalk", "email", "sms", "app"],
        "description": "通知渠道"
      },
      "recipients": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "接收人列表（用户ID或手机号）"
      },
      "title": {
        "type": "string",
        "description": "通知标题"
      },
      "content": {
        "type": "string",
        "description": "通知内容"
      },
      "priority": {
        "type": "string",
        "enum": ["low", "normal", "high", "urgent"],
        "default": "normal",
        "description": "优先级"
      }
    },
    "required": ["channel", "recipients", "title", "cont
  },
  "returns": {
    "type": "object",
    "properties": {
```

```
        "success": {
          "type": "boolean",
          "description": "是否发送成功"
        },
        "sent_count": {
          "type": "integer",
          "description": "成功发送人数"
        },
        "failed_count": {
          "type": "integer",
          "description": "发送失败人数"
        }
      }
    },
    "errors": {
      "INVALID_CHANNEL": "不支持的通知渠道",
      "INVALID_RECIPIENTS": "接收人格式错误",
      "RATE_LIMITED": "发送频率超限"
    }
  }
```

MCP描述示例 - 生成分析报告

JSON

💡 **综合应用：** 这个工具结合了查询数据+可视化+生成报告三个能力，展示复杂业务场景的MCP描述。

```
{
  "name": "generate_analysis_report",
  "description": "【复杂工具示例】基于投资数据生成可视化的效益分
  "version": "2.0.0",
  "parameters": {
    "type": "object",
    "properties": {
      "data_source": {
        "type": "string",
        "description": "数据源ID或查询条件"
      },
      "report_type": {
        "type": "string",
        "enum": ["roi_analysis", "progress_report", "buc
        "description": "报告类型"
      },
      "chart_types": {
        "type": "array",
        "items": {
          "enum": ["bar", "line", "pie", "table"]
        },
        "description": "包含的图表类型"
      },
      "output_format": {
        "type": "string",
        "enum": ["pdf", "excel", "html", "markdown"],
        "default": "pdf",
        "description": "输出格式"
      },
      "time_range": {
        "type": "object",
        "properties": {
          "start": {"type": "string", "format": "date"},
          "end": {"type": "string", "format": "date"}
        },
        "required": ["start", "end"]
      }
    },
    "required": ["data_source", "report_type"]
```

```
    },
    "returns": {
      "type": "object",
      "properties": {
        "report_url": {
          "type": "string",
          "description": "报告下载链接"
        },
        "preview_images": {
          "type": "array",
          "items": {"type": "string"},
          "description": "报告预览图（base64）"
        },
        "generated_at": {
          "type": "string",
          "format": "date-time",
          "description": "生成时间"
        }
      }
    }
  }
```

MCP的核心突破

"让工具具备"自描述"能力——工具自带说明书，AI能直接读懂能做什么、需要什么参数、返回什么结果"

Tools → Skills：AI行为指导

Skills是目录，不只是文件

Skills是**包含多个文件的目录**，不只是单个SKILL.md文件：

```
📁 skill-name/
  📄 SKILL.md
  📁 scripts/
    🐍 api.py
    🐍 process.py
  📁 references/
    📄 api-reference.md
    📄 examples.md
  📁 templates/
    📄 report-template.md
```

渐进式披露（Progressive Disclosure）

L1 Metadata（~100 words）

始终在上下文中

↓

L2 SKILL.md Body（<200行）

触发时加载

↓

L3 references/scripts/

按需加载

Skills定义示例 - 投资效益分析

YAML

```
# ==============================================
# SKILL定义文件 - 投资效益分析专家
# 作用：告诉AI"你是谁"、"你能做什么"、"怎么做"
# ==============================================

---
# Skill名称：使用kebab-case命名，要简洁明了
name: investment-analyzer

# 功能描述：这是最关键的部分！
# AI根据这段描述来决定什么时候调用这个Skill
description: "投资效益分析专家，计算项目ROI并生成评估报告。当用

# 触发方式：on_demand表示由AI判断何时使用
trigger: on_demand

# 标签：用于分类和检索
tags: investment, roi, analysis
---

# 【角色定义】告诉AI扮演什么角色
你是一位电力行业投资效益分析专家。你的任务是帮助用户分析投资项目的效

## 分析流程
# 关键教学点：这就是典型的"思考-规划-执行"流程！
1. 首先了解用户要分析的项目范围和目标
2. 使用query_investment_data工具查询相关项目数据
3. 使用execute_code工具进行ROI计算和数据处理
4. 根据计算结果评估项目等级（优秀/良好/需关注）
5. 使用generate_chart工具生成可视化图表
6. 生成包含图表和文字说明的完整分析报告

## 评估标准
- ROI > 15%: 优秀
- ROI 8-15%: 良好
- ROI < 8%: 需关注

## 详细参考（按需加载）
- 计算公式详解: references/roi_formula.md
```

```
- 行业基准数据: references/industry_benchmarks.md
- 示例报告: references/example_reports/

## 可执行脚本（按需调用）
- 复杂ROI计算: scripts/roi_calculator.py
- 数据验证: scripts/data_validator.py

## 注意事项
# ⚠️ 教学重点：这些约束条件确保AI不会"瞎说"
- 始终验证数据的完整性和准确性
- 如果发现异常数据，主动向用户确认
- 报告要包含具体的改进建议，不只是数据罗列

## 输出格式
你的回复应该包含：
1. 执行摘要（关键发现）
2. 详细分析（数据+解释）
3. 可视化图表
4. 改进建议
```

Skills定义示例 - 项目进度追踪

YAML

```
# ==========================================
# SKILL定义文件 - 项目进度追踪专家
# 作用：监控项目状态、预警风险、生成进度报告
# 对比上一个：同样是分析，但关注"时间"而非"金钱"
# ==========================================

---
name: project-tracker

# 描述中的触发词很重要！AI靠这些词识别用户意图
description: "追踪项目进度，识别延期风险。当用户询问'项目进度如何

trigger: on_demand
tags: project, tracking, management
---

# 【角色定义】
你是一位项目管理专家，专门负责追踪和监控项目进度。你的任务是帮助用户

## 工作流程
# 注意第5步：发现风险→主动通知，这就是Agent的"自主性"体现！
1. 了解用户要监控的项目范围和关注点
2. 使用get_project_status工具查询项目当前状态
3. 使用calculate_delay_risk工具评估延期风险
4. 识别关键里程碑的完成情况
5. 如果发现延期风险，使用send_notification发送预警  # ← 自主决
6. 生成包含进度图表和风险分析的报告

## 风险等级定义
- 正常：进度在计划范围内
- 延期风险：预计延期7天内
- 已延期：已经延期超过7天
- 提前完成：进度超前于计划

## 详细参考（按需加载）
- 项目状态定义：references/status_definitions.md
- 风险管理指南：references/risk_management.md

## 可执行脚本（按需调用）
```

```
- 进度计算: scripts/progress_calculator.py
- 风险预测: scripts/risk_predictor.py

## 输出内容
1. 项目整体进度百分比
2. 关键里程碑完成情况
3. 识别的风险项及等级
4. 具体的改进建议
5. 进度趋势图表
```

Skills定义示例 - 智能工单处理

YAML

```
# ==========================================
# SKILL定义文件 - 智能工单处理助手
# 作用：自动化处理重复性工作（分类→分配→跟踪）
# 核心价值：把人从繁琐工作中解放出来
# ==========================================

---
name: workorder-processor

# 描述中列出了完整的能力链：分类→诊断→推荐
description: "智能处理故障工单，自动分类、诊断、推荐解决方案。当

trigger: on_demand
tags: workorder, diagnosis, support
---

# 【角色定义】
你是一位智能工单处理专家。你的任务是自动识别工单类型、智能分配给合适

## 处理流程
1. 接收工单内容，分析工单描述和附件
2. 使用classify_workorder工具识别工单类型（故障/咨询/投诉/建议
3. 根据工单类型和紧急程度，使用find_best_handler工具找到最合适的
4. 使用generate_reply工具生成专业的自动回复
5. 使用create_workorder工具创建正式工单并分配
6. 持续跟踪处理进度，超时自动升级

## 工单优先级定义
# 教学点：这就是"决策树"的实际应用！
# Agent根据这些规则自动判断优先级，不需要人干预
- urgent：紧急故障，影响业务运行     # → 立即分配，短信通知
- high：重要问题，需要尽快处理       # → 2小时内响应
- medium：一般问题，按正常流程处理   # → 24小时内处理
- low：咨询建议类，可延后处理         # → 3天内回复即可

## 分配原则
- 根据工单类型匹配专业技能
- 考虑处理人当前工作负荷
- 优先分配给历史处理效率高的员工
```

```
## 详细参考（按需加载）
- 工单分类标准: references/workorder_classification.md
- 处理人技能矩阵: references/skill_matrix.md

## 可执行脚本（按需调用）
- 智能分类: scripts/auto_classifier.py
- 负载均衡: scripts/workload_balancer.py

## 输出内容
1. 识别的工单类型和子类型
2. 分配的处理人及理由
3. 自动回复内容
4. 预计处理时长
5. 工单编号和跟踪链接
```

完整技术栈架构

决策层

RPAR循环

思考 → 规划 → 执行 → 反思

Agent的"大脑"

能力层

Skills

查询投资效益 | 生成报告 | 分析ROI

AI行为指导

实现层

Code Tools + Tools

自定义代码 | 原子能力

技术实现

协议层

MCP

标准化描述和调用

AI能理解

基础层

API

外部系统原始接口

能力源头

课程总结

RPAR循环：核心框架

🧠

思考

理解任务现状

📋

规划

制定执行步骤

⚡

执行

调用工具完成

🔄

反思

评估调整策略

三大核心收获

1

Agent本质

从"动口"到"动手"，从"顾问"到"助手"

2

RPAR框架

思考-规划-执行-反思的认知循环

3

工具演进

API→MCP→Tools→Skills，让AI能理解、执行并按指导工作

"完整的Agentic AI = RPAR大脑 + Skills行为指导 + MCP标准 + API手脚"

下节课预告

Day 2

Agent开发实战

在灵知平台上实现RPAR框架

敬请期待

Day 1: Agentic AI 设计范式 | 国网四川电力AI种子团队培训

2026.03 | RPAR认知架构与工程实现